



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

FPGA Design Using VHDL

Module 3D: Counters in VHDL

Counters in VHDL



- Counters can be designed using flip flops
- However, we can also describe them behaviorally in VHDL
 - Let the synthesis tool implement counters at the flip-flop level
 - Easier to maintain and update depending on needs
- Counters are very useful in sequential designs and have lots of applications

Up Counter in VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.all;

entity upCounter is port (
    clk      : in std_logic;
    reset    : in std_logic;
    counter  : out unsigned(7 downto 0));
end upCounter;

architecture rtl of upCounter is
    signal cntr_sig : unsigned(7 downto 0); ←
begin

    process(clk, reset)
    begin
        → if(reset = '1') then
            cntr_sig <= (others=> '0');
        → elsif(rising_edge(clk)) then
            cntr_sig <= cntr_sig + 1;
        end if;
    end process;
    → counter <= cntr_sig;

end rtl;
```

```
process(clk, reset)
begin
    if (reset = '1') then
        Q <= '0';
    elsif(rising_edge(clk)) then
        Q <= D;
    end if;
end process;
```

Down Counters in VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.all;

entity downCounter is port (
    clk      : in std_logic;
    reset    : in std_logic;
    counter  : out unsigned(7 downto 0));
end downCounter;

architecture rtl of downCounter is
    signal cnt_sig : unsigned(7 downto 0);
begin

    process(clk, reset)
    begin
        if(reset = '1') then
            cnt_sig <= (others=> '1');
        elsif(rising_edge(clk)) then
            cnt_sig <= cnt_sig - 1;
        end if;
    end process;
    counter <= cnt_sig;

end rtl;
```

```
process(clk, reset)
begin
    if (reset = '1') then
        Q <= '0';
    elsif(rising_edge(clk)) then
        Q <= D;
    end if;
end process;
```

Pulses in VHDL



- Example use for using a counter
 - Pulse that goes high every time 8-bit counter is zero
- Characteristic of pulses
 - Periodic
 - High for one clock period, low otherwise
 - If using 100MHz clock: pulse is high for 10ns and periodic with period of 2.56us
 - Allows pulse to be used as enable for single event

Counters with Enable

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.all;

entity CounterWithEn is port (
    clk      : in std_logic;
    reset    : in std_logic;
    enable   : in std_logic;
    counter  : out unsigned(7 downto 0));
end CounterWithEn;

architecture rtl of CounterWithEn is
    signal cnt_sig : unsigned(7 downto 0);
begin

    process(clk, reset)
    begin
        if(reset = '1') then
            cnt_sig <= (others=> '0');
        elsif(rising_edge(clk)) then
            if(enable = '1') then
                cnt_sig <= cnt_sig + 1;
            end if;
        end if;
    end process;
    counter <= cnt_sig;

end rtl;
```

```
process(clk, reset)
begin
    if (reset = '1') then
        Q <= '0';
    elsif (rising_edge(clk)) then
        if (e = '1') then
            Q <= D;
        end if;
    end if;
end process;
```

Putting it All Together

```
-- entity omitted to save space
```

```
architecture rtl of counterWithEnable is
  signal cntr8bit : unsigned(7 downto 0);
  signal pulse : std_logic;
  signal slowCntr16bit : unsigned(15 downto 0);
```

```
begin
```

```
-- 8 bit counter that increments by 1 every 10ns (using 100 MHz clock)
```

```
upCounter_inst : entity upCounter port map (
  clk => clk_100MHz,
  reset => BTN0,
  counter => cntr8bit);
```

```
pulse <= '1' when (cntr8bit = 0) else '0'; -- pulse hi for 10ns, period 2.56us
```

```
-- 16 bit counter that increments every 2.56us using pulse from above
```

```
process(clk_100MHz, BTN0)
begin
  if(BTN0= '1') then
    slowCntr16bit <= (others=> '0');
  elsif(rising_edge(clk_100MHz)) then
    if(pulse = '1') then
      slowCntr16bit <= slowCntr16bit + 1;
    end if;
  end if;
end process;
```

```
end rtl;
```

Timing Diagram

